2025.05.27

PROGRAMMING LANGUAGES AND COMPILERS

PROGRAMMING ASSIGNMENT #1

411221420 411221425

資工二 資工二

林彥宏 陸竑宇

II. PROBLEM DESCRIPTION (1/2)

The objective of this assignment is to develop the front end of a compiler for the T programming language using Lex (Flex) and Yacc (Bison) tools. This front end is responsible for two main components:

1. Lexical Analyzer: Written in t_lex.l, this component tokenizes the input source code into valid tokens based on the T language lexicon.
2. Syntax Analyzer: Written in t_parse.y, this parser verifies whether the token stream follows the grammar rules of the T language, as defined in EBNF. The parser also prints applied grammar rules during parsing.

II. PROBLEM DESCRIPTION (2/2)

The system must be split into four files:

t_lex.l: Contains the Lex specification to define token patterns.

t_parse.y: Contains the Yacc specification to define grammar rules and parsing logic.

t2c.c: The main C program that initiates the parser.

t2c.h: A header file for shared declarations between components.

III. HIGHLIGHT OF THE WAY I WRITE THE PROGRAM (1/3)

T_PARSE.Y

Based on the EBNF grammar provided in the homework, the parser rules include:

- High-level structure (Program -> MethodDecl MethodDecl*)
- Expressions, conditional statements, loops
- Each rule includes a printf() statement to output the rule name during parsing.

EXPRESSION

- Expressions are parsed using a recursive and precedence aware approach that distinguishes between additive and multiplicative levels. This structure correctly reflects operator precedence
- At the bottom level, primaryExpr handles:

1. Literal values (integers, reals)
2. Variables (identifiers)
3. Grouped expressions in parentheses
4. Function calls with actual parameters

- This setup ensures the parser can handle complex arithmetic expressions and nested calls accurately, while also making the grammar extensible for future semantic analysis or AST construction.

STATEMENT

- The statement rule covers all the major control structures and operations defined in the T language, ensuring that the parser can recognize and handle various kinds of program logic.
- Each of the subrules is implemented in its own production rule to maintain modularity and clarity.
- Each rule includes a printf() call that reports which grammar rule was applied during parsing, which aids in debugging and traceability.

III. HIGHLIGHT OF THE WAY I WRITE THE PROGRAM (2/3)

T_LEX.L

In the lexical analyzer (t_lex.l), identifiers and numeric constants are processed using regular expressions and mapped to appropriate token types. Below are the key token rules:

IDENTIFIERS

- Pattern {ID} matches an identifier: a letter followed by letters or digits ([A-Za-z][A-Za-z0-9]*).
- When matched:

1. The matched text (lexeme) is copied into a global character array name.
2. The token lID is returned to the parser.

- Purpose: Used for variable names, function names, and parameters in the T language.

INTEGER NUMBERS

- Pattern {DIG} matches one or more digits ([0-9][0-9]*).
- When matched:

1. atoi() converts the string into an integer.
2. The result is stored in a global variable ival.
3. The token lINUM is returned to the parser.

- Purpose: Allows the parser to recognize and evaluate integer constants.

REAL NUMBERS

- Pattern {RNUM} matches a real number format like 123.456.
- When matched:

1. atof() converts the string to a float/double.
2. The result is stored in the global variable rval.
3. The token lRNUM is returned to the parser.

- Purpose: Supports real number literals in arithmetic expressions.

III. HIGHLIGHT OF THE WAY I WRITE THE PROGRAM (3/3)

GENERATING REQUIRED FILES

To compile and run the front-end compiler for the T language, I used Flex, Bison, and GCC. The build process is organized into the following steps:

1. GENERATE THE PARSER WITH BISON:
   BISON -D -O T_PARSE.C T_PARSE.Y
2. GENERATE THE LEXER WITH FLEX:
   FLEX -O T_LEX.C T_LEX.L
3. COMPILE ALL SOURCE FILES USING GCC
   GCC -C T_PARSE.C # GENERATES T_PARSE.O
   GCC -C T_LEX.C # GENERATES T_LEX.O
   GCC -C T2C.C # GENERATES T2C.O
4. LINK ALL OBJECT FILES INTO THE FINAL EXECUTABLE:
   GCC -O PARSE T2C.O T_LEX.O T_PARSE.O
5. RUN THE PROGRAM:
   ./PARSE SAMPLE.T

t_parse.y:

```
%{
    #include <stdio.h>
    #include <stdlib.h>
    #include "t2c.h"
    #include "t_parse.h"
%}
/*This section is enclosed by %{ ... %} and is copied verbatim into the top of the
generated .c parser file.
It includes necessary C headers and your custom headers (t2c.h, t_parse.h).
These headers likely contain function declarations, data types, or utility functions
needed during parsing or for semantic actions.*/
```

```
%token lWRITE lREAD lIF lASSIGN
%token lRETURN lBEGIN lEND
%token lEQU lNEQ lGT lLT lGE lLE
%token lADD lMINUS
%token lTIMES lDIVIDE
%token lLP lRP
%token lINT lREAL lSTRING
%token lELSE
%token lMAIN
%token lSEMI lCOMMA
%token lID lINUM lRNUM lQSTR
%token lAND lOR lNOT
/*These %token declarations define the terminal symbols (tokens) that the parser can
accept. These correspond to lexical tokens provided by the lexer (Flex).
Examples:
lIF, lWRITE, lRETURN are reserved keywords.
lID, lINUM, lRNUM, lQSTR are identifiers, integers, real numbers, and quoted strings.
lADD, lMINUS, lASSIGN represent operators like +, -, and =.
These token names allow your grammar to be abstracted from the actual text
representation, which is defined in the lexer.*/
```

```
%left  lOR
%left  lAND
%left  lEQU lNEQ lGT lLT lGE lLE
%left  lADD lMINUS
%left  lTIMES lDIVIDE
%right UMINUS
%right lNOT
/*This section resolves ambiguities in the grammar by assigning precedence and
associativity to operators:
*  lOR and lAND are logical operators with low precedence.
*  lADD, lMINUS have medium precedence.
*  lTIMES, lDIVIDE have higher precedence.
*  UMINUS is a unary minus operator (like -x), and it’s right-associative.
*  lNOT is a logical negation, also right-associative.
The %prec UMINUS in a rule explicitly gives a production the precedence of UMINUS.*/
```

```
%expect 1 //to acknowledge known grammar ambiguity while keeping other conflicts
visible
```

```
%%
prog    :    mthdcls
        { printf("Program -> MethodDecls\nParsed OK!\n"); }
    |
        { printf("****** Parsing failed!\n"); }
    ;
/*This is the starting point of the grammar.
If mthdcls (method declarations) are parsed, it prints success.
Otherwise, it prints a failure message.*/
```

```
mthdcls     :    mthdcl mthdcls
        { printf("MethodDecls -> MethodDecl MethodDecls\n"); }
    |    mthdcl
        { printf("MethodDecls -> MethodDecl\n"); }
    ;

type     :    lINT
        { printf("Type -> INT\n"); }
    |    lREAL
        { printf("Type -> REAL\n"); }
    ;

mthdcl     :    type lMAIN lID lLP formals lRP block
        { printf("MethodDecl -> Type MAIN ID LP Formals RP Block\n"); }
    |    type lID lLP formals lRP block
        { printf("MethodDecl -> Type ID LP Formals RP Block\n"); }
    ;

formals     :    formal oformal
        { printf("Formals -> Formal OtherFormals\n"); }
    |
        { printf("Formals -> (empty)\n"); }
    ;

formal     :    type lID
        { printf("Formal -> Type ID\n"); }
    ;

oformal     :    lCOMMA formal oformal
        { printf("OtherFormals -> COMMA Formal OtherFormals\n"); }
    |
        { printf("OtherFormals -> (empty)\n"); }
    ;
/*1. mthdcls is a list of method declarations.
2. mthdcl defines a method, with or without the main keyword.
Syntax like int main() { ... } or real func(int x) { ... }
3. type represents data types: int or real.
4. formals is the parameter list in function declarations.
Supports zero or more parameters using recursive rules.*/
```

```
block : lBEGIN statements lEND
        { printf("Block -> BEGIN Statements END\n"); }
    ;

statements : statement statements
        { printf("Statements -> Statement Statements\n"); }
    |
        { printf("Statements -> (empty)\n"); }
    ;

statement : block
        { printf("Processing Statement: Block -> BEGIN Statements END\n"); }
    | localVarDecl
        { printf("Processing Statement: LocalVarDecl -> Type ID SEMI or Type
AssignStmt\n"); }
    | assignStmt
        { printf("Processing Statement: AssignStmt -> ID ASSIGN Expression SEMI\n"); }
    | returnStmt
        { printf("Processing Statement: ReturnStmt -> RETURN Expression SEMI\n"); }
    | ifStmt
        { printf("Processing Statement: IfStmt -> IF LP BoolExpr RP Statement [ELSE
Statement]\n"); }
    | writeStmt
        { printf("Processing Statement: WriteStmt -> WRITE LP Expression COMMA QString
RP SEMI\n"); }
    | readStmt
        { printf("Processing Statement: ReadStmt -> READ LP ID COMMA QString RP
SEMI\n"); }
    | error lSEMI
        { yyerrok; printf("Recovered from error in statement.\n"); }
    ;
/*A block is enclosed in BEGIN...END.
A block contains zero or more statements.
statement supports:
*  Code blocks
*  Variable declarations
*  Assignments
*  Returns
*  Conditionals
*  I/O statements
Each statement includes a debug printf() to indicate what is being parsed.*/
```

```
localVarDecl : type lID lSEMI
        { printf("Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI\n"); }
    | type assignStmt
        { printf("Processing LocalVarDecl: LocalVarDecl -> Type AssignStmt\n"); }
    ;

assignStmt : lID lASSIGN expression lSEMI
        { printf("Processing AssignStmt: AssignStmt -> ID ASSIGN Expression SEMI\n");
}
    ;

returnStmt : lRETURN expression lSEMI
        { printf("Processing ReturnStmt: ReturnStmt -> RETURN Expression SEMI\n"); }
    ;

ifStmt : lIF lLP boolExpr lRP statement
        { printf("Processing IfStmt: IfStmt -> IF LP BoolExpr RP Statement\n"); }
    | lIF lLP boolExpr lRP statement lELSE statement
        { printf("Processing IfStmt: IfStmt -> IF LP BoolExpr RP Statement ELSE
Statement\n"); }
    ;

writeStmt : lWRITE lLP expression lCOMMA lQSTR lRP lSEMI
        { printf("Processing WriteStmt: WriteStmt -> WRITE LP Expression COMMA QString
RP SEMI\n"); }
    ;

readStmt : lREAD lLP lID lCOMMA lQSTR lRP lSEMI
        { printf("Processing ReadStmt: ReadStmt -> READ LP ID COMMA QString RP
SEMI\n"); }
    ;
/*1. Variable Declaration: With or without initialization.
2. Assignment: E.g., x = 5;
3. Return Statement: E.g., return x + 1;
4. If/Else Statement: Nested blocks or single statements are upported.
5. Write/Read Statements: Output or input using expressions and string descriptions.*/
```

```
expression : multiplicativeExpr
        { printf("Processing Expression: Expression -> MultiplicativeExpr\n"); }
    | expression lADD multiplicativeExpr
        { printf("Processing Expression: Expression -> Expression ADD
MultiplicativeExpr\n"); }
    | expression lMINUS multiplicativeExpr
        { printf("Processing Expression: Expression -> Expression MINUS
MultiplicativeExpr\n"); }
    ;

multiplicativeExpr : unaryExpr
        { printf("Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr\n");
}
    | multiplicativeExpr lTIMES unaryExpr
        { printf("Processing MultiplicativeExpr: MultiplicativeExpr ->
MultiplicativeExpr TIMES UnaryExpr\n"); }
    | multiplicativeExpr lDIVIDE unaryExpr
        { printf("Processing MultiplicativeExpr: MultiplicativeExpr ->
MultiplicativeExpr DIVIDE UnaryExpr\n"); }
    ;

unaryExpr : primaryExpr
        { printf("Processing UnaryExpr: UnaryExpr -> PrimaryExpr\n"); }
    | lMINUS unaryExpr %prec UMINUS
        { printf("Processing UnaryExpr: UnaryExpr -> MINUS UnaryExpr\n"); }
    ;
/*Recursive definitions follow standard precedence hierarchy:
*  Additive (+, -)
*  Multiplicative (*, /)
*  Unary (-x)
These match standard arithmetic grammar.*/
```

```
primaryExpr : lINUM
        { printf("Processing PrimaryExpr: PrimaryExpr -> INUM\n"); }
    | lRNUM
        { printf("Processing PrimaryExpr: PrimaryExpr -> RNUM\n"); }
    | lID
        { printf("Processing PrimaryExpr: PrimaryExpr -> ID\n"); }
    | lLP expression lRP
        { printf("Processing PrimaryExpr: PrimaryExpr -> LP Expression RP\n"); }
    | lID lLP actualParams lRP
        { printf("Processing PrimaryExpr: PrimaryExpr -> ID LP ActualParams RP\n"); }
    | lQSTR
        { printf("Processing PrimaryExpr: PrimaryExpr -> QSTR\n"); }
    ;
//Base-level expressions include constants, variables, function calls, and
parenthesized expressions.

boolExpr : expression lEQU expression
        { printf("Processing BoolExpr: BoolExpr -> Expression EQU Expression\n"); }
    | expression lNEQ expression
        { printf("Processing BoolExpr: BoolExpr -> Expression NEQ Expression\n"); }
    | expression lGT expression
        { printf("Processing BoolExpr: BoolExpr -> Expression GT Expression\n"); }
    | expression lGE expression
        { printf("Processing BoolExpr: BoolExpr -> Expression GE Expression\n"); }
    | expression lLT expression
        { printf("Processing BoolExpr: BoolExpr -> Expression LT Expression\n"); }
    | expression lLE expression
        { printf("Processing BoolExpr: BoolExpr -> Expression LE Expression\n"); }
    | boolExpr lAND boolExpr
        { printf("Processing BoolExpr: BoolExpr -> BoolExpr AND BoolExpr\n"); }
    | boolExpr lOR boolExpr
        { printf("Processing BoolExpr: BoolExpr -> BoolExpr OR BoolExpr\n"); }
    | lNOT boolExpr
        { printf("Processing BoolExpr: BoolExpr -> NOT BoolExpr\n"); }
    ;
/*Logical operations: AND, OR, NOT
Relational comparisons: ==, !=, <, >, <=, >=
These can be used in conditions such as if (...).*/

actualParams : expression
        { printf("Processing ActualParams: ActualParams -> Expression\n"); }
    | actualParams lCOMMA expression
        { printf("Processing ActualParams: ActualParams -> ActualParams COMMA
Expression\n"); }
    |
        { printf("Processing ActualParams: ActualParams -> (empty)\n"); }
    ;
//Handles zero or more expressions as arguments in a function call.
%%
```

```c
int yyerror(char *s)
{
    printf("Syntax Error: %s\n", s);
    return 1;
}
/*Called when a syntax error occurs (automatically by Bison).
Outputs a human-readable message and returns non-zero.*/
```

t_lex.l:

```
%{
#include "t2c.h"
#include "t_parse.h"
#include <string.h>
%}
/*```
This block is copied directly to the top of the generated .c file.
Includes:
*  t2c.h and t_parse.h: likely define tokens and global variables (name, ival, rval,
qstr).
*  string.h: for functions like strcpy() and sscanf().*/
```

```
%x C_COMMENT
//Declares a start condition named C_COMMENT to handle block comments (/* ... */).
//This enables the lexer to switch context when entering exiting comment mode.

ID  [A-Za-z][A-Za-z0-9]* //Identifiers must start with a letter and be followed by
alphanumeric characters.
DIG [0-9][0-9]* //One or more digits.
RNUM {DIG}"."{DIG} //Real number format (e.g. 123.45).
NQUO [^"] //Any character except a double quote; used for string matching.
```

```
%%
WRITE        {return lWRITE;}
READ        {return lREAD;}
IF        {return lIF;}
ELSE        {return lELSE;}
RETURN        {return lRETURN;}
BEGIN        {return lBEGIN;}
END        {return lEND;}
MAIN        {return lMAIN;}
INT        {return lINT;}
REAL        {return lREAL;}
/*These match exact keywords in uppercase.
The rule order matters: keywords are checked before general identifiers.*/
```

```
";"        {return lSEMI;}
","        {return lCOMMA;}
"("        {return lLP;}
")"        {return lRP;}
"+"        {return lADD;}
"-"        {return lMINUS;}
"*"        {return lTIMES;}
"/"        {return lDIVIDE;}
">"        {return lGT;}
"<"        {return lLT;}
":="        {return lASSIGN;}
"=="        {return lEQU;}
"!="        {return lNEQ;}
">="        {return lGE;}
"<="        {return lLE;}
/*These match punctuation, arithmetic operators, and relational operators.
Tokens like ":=" must be listed before the single character ":", if it existed, to
avoid premature matches.*/
```

```
{ID}    { strcpy(name, yytext); return lID; }
{DIG}   { ival = atoi(yytext); return lINUM; }
{RNUM}  { rval = atof(yytext); return lRNUM; }
/*Saves the matched text into global variables for use by the parser:
name: for identifiers.
ival: for integers.
rval: for real numbers.*/
```

```
\"{NQUO}*\"    {sscanf(yytext,"%s", qstr); return lQSTR;}
//Matches quoted strings ("...") with no embedded quotes.
//sscanf() is used to copy the content into qstr.

"/*"        { BEGIN(C_COMMENT); } //switches the scanner to C_COMMENT mode.
<C_COMMENT>"*/"    { BEGIN(INITIAL); }
<C_COMMENT>\n    { }
<C_COMMENT>.    { }
/*In C_COMMENT:
When it sees * /, it returns to the normal scanning mode.
All other characters (including newlines) are ignored.*/

[ \t\n]        {}
.        { printf("Unknown character: %s\n", yytext); }
/*Skips whitespace.
If any unknown character appears, it prints a message.*/
```

```
%%
int yywrap() {return 1;} //Returns 1 to signal end of file and stop scanning.

void print_lex( int t ) {
    switch( t ) {
    case lWRITE: printf("WRITE\n");
        break;
    case lREAD: printf("READ\n");
        break;
    case lIF: printf("IF\n");
        break;
    case lELSE: printf("ELSE\n");
        break;
    case lRETURN: printf("RETURN\n");
        break;
    case lBEGIN: printf("BEGIN\n");
        break;
    case lEND: printf("END\n");
        break;
    case lMAIN: printf("MAIN\n");
        break;
    case lSTRING: printf("STRING\n");
        break;
    case lINT: printf("INT\n");
        break;
    case lREAL: printf("REAL\n");
        break;
    case lSEMI: printf("SEMI\n");
        break;
    case lCOMMA: printf("COMMA\n");
        break;
    case lLP: printf("LP\n");
        break;
    case lRP: printf("RP\n");
        break;
    case lADD: printf("ADD\n");
        break;
    case lMINUS: printf("MINUS\n");
        break;
    case lTIMES: printf("TIMES\n");
        break;
    case lDIVIDE: printf("DIVIDE\n");
        break;
    case lASSIGN: printf("ASSIGN\n");
        break;
    case lEQU: printf("EQU\n");
        break;
    case lNEQ: printf("NEQ\n");
        break;
    case lID: printf("ID(%s)\n", name);
```

```
        break;
    case lINUM: printf("INUM(%d)\n", ival);
        break;
    case lRNUM: printf("RNUM(%f)\n", rval);
        break;
    case lQSTR: printf("QSTR(%s)\n", qstr);
        break;
    default: printf("******** lexical error!!!");
    }
}
/*Utility for debugging.
After the lexer returns a token, print_lex() prints a readable description of that
token.
For identifiers, numbers, and strings, it also prints their value.*/
```

Test run results

1. test.t with z := f2(x,y) + f2(y,x) * f2(x,y);

```
Type -> INT
Type -> INT
Formal -> Type ID
Type -> INT
Formal -> Type ID
OtherFormals -> (empty)
OtherFormals -> COMMA Formal OtherFormals
Formals -> Formal OtherFormals
Type -> INT
Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI
Processing Statement: LocalVarDecl -> Type ID SEMI or Type AssignStmt
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> MultiplicativeExpr TI
Processing Expression: Expression -> MultiplicativeExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> MultiplicativeExpr TI
Processing Expression: Expression -> Expression MINUS MultiplicativeExpr
Processing AssignStmt: AssignStmt -> ID ASSIGN Expression SEMI
Processing Statement: AssignStmt -> ID ASSIGN Expression SEMI
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ReturnStmt: ReturnStmt -> RETURN Expression SEMI
Processing Statement: ReturnStmt -> RETURN Expression SEMI
Statements -> (empty)
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Block -> BEGIN Statements END
MethodDecl -> Type ID LP Formals RP Block
Type -> INT
Formals -> (empty)
Type -> INT
Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI
Processing Statement: LocalVarDecl -> Type ID SEMI or Type AssignStmt
Processing ReadStmt: ReadStmt -> READ LP ID COMMA QString RP SEMI
Processing Statement: ReadStmt -> READ LP ID COMMA QString RP SEMI
Type -> INT
Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI
Processing Statement: LocalVarDecl -> Type ID SEMI or Type AssignStmt
Processing ReadStmt: ReadStmt -> READ LP ID COMMA QString RP SEMI
Processing Statement: ReadStmt -> READ LP ID COMMA QString RP SEMI
Type -> INT
Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI
Processing Statement: LocalVarDecl -> Type ID SEMI or Type AssignStmt
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> Expression
```

```
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> ActualParams COMMA Expression
Processing PrimaryExpr: PrimaryExpr -> ID LP ActualParams RP
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> Expression
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> ActualParams COMMA Expression
Processing PrimaryExpr: PrimaryExpr -> ID LP ActualParams RP
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> Expression ADD MultiplicativeExpr
Processing AssignStmt: AssignStmt -> ID ASSIGN Expression SEMI
Processing Statement: AssignStmt -> ID ASSIGN Expression SEMI
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing WriteStmt: WriteStmt -> WRITE LP Expression COMMA QString RP SEI
Processing Statement: WriteStmt -> WRITE LP Expression COMMA QString RP SEI
Statements -> (empty)
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Block -> BEGIN Statements END
MethodDecl -> Type MAIN ID LP Formals RP Block
MethodDecls -> MethodDecl
MethodDecls -> MethodDecl MethodDecls
Program -> MethodDecls
Parsed OK!
```

2. test.t with z := f2(x,y) + f2(y,x);

```
Type -> INT
Type -> INT
Formal -> Type ID
Type -> INT
Formal -> Type ID
OtherFormals -> (empty)
OtherFormals -> COMMA Formal OtherFormals
Formals -> Formal OtherFormals
Type -> INT
Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI
Processing Statement: LocalVarDecl -> Type ID SEMI or Type AssignStmt
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> MultiplicativeExpr TI
Processing Expression: Expression -> MultiplicativeExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> MultiplicativeExpr TI
Processing Expression: Expression -> Expression MINUS MultiplicativeExpr
Processing AssignStmt: AssignStmt -> ID ASSIGN Expression SEMI
Processing Statement: AssignStmt -> ID ASSIGN Expression SEMI
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ReturnStmt: ReturnStmt -> RETURN Expression SEMI
Processing Statement: ReturnStmt -> RETURN Expression SEMI
Statements -> (empty)
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Block -> BEGIN Statements END
MethodDecl -> Type ID LP Formals RP Block
Type -> INT
Formals -> (empty)
Type -> INT
Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI
Processing Statement: LocalVarDecl -> Type ID SEMI or Type AssignStmt
Processing ReadStmt: ReadStmt -> READ LP ID COMMA QString RP SEMI
Processing Statement: ReadStmt -> READ LP ID COMMA QString RP SEMI
Type -> INT
Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI
Processing Statement: LocalVarDecl -> Type ID SEMI or Type AssignStmt
Processing ReadStmt: ReadStmt -> READ LP ID COMMA QString RP SEMI
Processing Statement: ReadStmt -> READ LP ID COMMA QString RP SEMI
Type -> INT
Processing LocalVarDecl: LocalVarDecl -> Type ID SEMI
Processing Statement: LocalVarDecl -> Type ID SEMI or Type AssignStmt
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> Expression
```

```
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> ActualParams COMMA Expression
Processing PrimaryExpr: PrimaryExpr -> ID LP ActualParams RP
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> Expression
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> ActualParams COMMA Expression
Processing PrimaryExpr: PrimaryExpr -> ID LP ActualParams RP
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> Expression
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing ActualParams: ActualParams -> ActualParams COMMA Expression
Processing PrimaryExpr: PrimaryExpr -> ID LP ActualParams RP
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> MultiplicativeExpr TI
Processing Expression: Expression -> Expression ADD MultiplicativeExpr
Processing AssignStmt: AssignStmt -> ID ASSIGN Expression SEMI
Processing Statement: AssignStmt -> ID ASSIGN Expression SEMI
Processing PrimaryExpr: PrimaryExpr -> ID
Processing UnaryExpr: UnaryExpr -> PrimaryExpr
Processing MultiplicativeExpr: MultiplicativeExpr -> UnaryExpr
Processing Expression: Expression -> MultiplicativeExpr
Processing WriteStmt: WriteStmt -> WRITE LP Expression COMMA QString RP SE
Processing Statement: WriteStmt -> WRITE LP Expression COMMA QString RP SE
Statements -> (empty)
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Statements -> Statement Statements
Block -> BEGIN Statements END
MethodDecl -> Type MAIN ID LP Formals RP Block
MethodDecls -> MethodDecl
MethodDecls -> MethodDecl MethodDecls
Program -> MethodDecls
Parsed OK!
```

VI. DISCUSSION

This project involved building the front-end of a compiler for the T programming language using Flex and Bison. Through this assignment, I learned how to define a lexical analyzer using Flex and how to write grammar rules for syntax analysis using Bison. By separating the implementation into four files—t_lex.l, t_parse.y, t2c.c, and t2c.h, the structure of the compiler front-end became clear and modular.

During development, I faced challenges in writing correct regular expressions and managing the complexity of grammar rules, especially those involving nested statements and operator precedence. However, by using step-by-step testing and carefully analyzing the sample program, I was able to resolve these issues effectively.

This assignment enhanced my understanding of compiler design and helped me gain practical experience with industry-standard tools. It was a rewarding exercise that deepened my appreciation for the role of lexical and syntax analysis in programming languages.